

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 2 — Misurare e migliorare

Tre prompt, due cicli

Tre prompt, due cicli

Il laboratorio del Modulo 2, l'esercitazione M2-E02: obiettivo, tempo, due percorsi

- **Obiettivo**: tre prompt di lavoro di una riga, uno per famiglia — **sviluppo**, **supporto**, **sistemi** — migliorati come si migliora un programma: **due cicli** ciascuno, misurare, cambiare **una cosa sola**, rimisurare
- **Tempo**: 80 minuti — cinque per partire, poi circa 25 per prompt
- **Percorso A**, i vostri prompt: in una o più famiglie, un prompt che usate davvero, con i dati sostituiti da **valori inventati** — niente nomi di clienti, password, indirizzi o configurazioni reali. Il prompt in `prompt/v0.md`, i dati in `dati/`, in `rubrica.md` quattro o cinque affermazioni che una buona risposta rispetta, e in `controlli.tsv` almeno un controllo adatto al vostro compito
- **Percorso B**, i prompt di CiaspoLink, l'operatore inventato dei progetti di esempio: le tre famiglie così come sono — la prova del docente è su queste
- Ogni versione misurata ha la sua riga nel **registro**, `registro.md`: che cosa avete cambiato, che cosa è successo
- Nel repository delle esercitazioni, il `README.md` di `m2_prompting_avanzato/s2_misurare_e_migliorare/m2_e3_tre_prompt_due_cicli/M2-E02_tre_prompt_due_cicli/`

Due cicli su tre prompt reali valgono più di dieci tecniche lette.

I tre prompt di partenza

Percorso B: un prompt di una riga per famiglia, con i suoi dati e la sua rubrica

Il prompt, `prompt/v0.md`	**I dati**, in `dati/`, **e la rubrica**, `rubrica.md`

</> Sviluppo
`spiega questa funzione`
`{{codice}}`
	Un frammento del gestionale in PHP, che calcola il canone di una linea, e la convenzione del team sul rateo
	La rubrica, 5 affermazioni: dove il calcolo **sbaglia** rispetto alla convenzione, con un caso concreto, e la **correzione**

Supporto
`rispondi al cliente`
`{{messaggio}}`
	L'e-mail di un cliente privato — la linea cade più volte all'ora — e la guida al tono di voce dell'assistenza
	La rubrica, 5 affermazioni: il «lei», **passi concreti**, nessun orario promesso, al più una scusa, il **tono** della guida

Sistemi
`cosa è successo?`
`{{log}}`
	Un estratto del syslog di una OLT, `olt-pergine-01`
	La rubrica, 4 affermazioni: **un guasto solo**, a monte; quanti clienti; il **primo controllo**; le cause come ipotesi

Fra **doppie graffe**, un segnaposto: `misura.sh` ci mette il file di `dati/` con quel nome. In ogni cartella anche `controlli.tsv`: i controlli scritti in codice

Un ciclo, passo per passo

In Claude Code, con il punto esclamativo: qui su sistemi, al suo posto la vostra famiglia

```
! ./misura.sh sistemi v0
! cp sistemi/prompt/v0.md sistemi/prompt/v1.md
# in v1.md cambiate una cosa sola, poi:
! ./misura.sh sistemi v1
! ./misura.sh --confronta sistemi v0 v1
! git diff --no-index sistemi/prompt/v0.md sistemi/prompt/v1.md
# e una riga in registro.md; il secondo ciclo, dalla migliore:
! cp sistemi/prompt/v1.md sistemi/prompt/v2.md
```

- Con il **punto esclamativo** davanti il comando va **dritto alla shell**, senza passare da Claude e senza chiedere approvazione; comando e output entrano nella conversazione, e Claude li può leggere con voi
- `misura.sh` fa tre passi e li stampa: la **risposta** (`claude -p` su Sonnet, effort low, nessuno strumento), i **controlli** di `controlli.tsv`, il **giudice** — un subagent su Haiku che, per ogni affermazione della rubrica, scrive **prima il motivo, poi l'esito**
- **Una cosa sola** per versione: il **testo** (un ingrediente del template), il **modello** (`--modello haiku` o `--modello opus`) o l'**effort** (`--effort medium`). La verifica vuole `v1.md` diverso da `v0.md`: nel primo ciclo, **il testo**
- `--confronta`: controlli, giudizi e token delle due versioni, **fianco a fianco**; `git diff --no-index`: il testo cambiato. Poi una riga nel **registro** — e il secondo ciclo riparte dalla **versione migliore**

Laboratorio: primo prompt

La famiglia più vicina al vostro lavoro: due cicli, fino a v2

```
# dalla cartella esercitazioni
tools/prepara_esercizio.sh M2-E02
cd ~/corso-claude-code/M2-E02
claude --model sonnet
```

1. Scegliete il percorso, A o B, e la famiglia **più vicina al vostro lavoro**
2. Primo ciclo: la misura di `v0`; `v1` con **il testo** cambiato, la sua misura, il confronto; una riga nel registro
3. Secondo ciclo, dalla **versione migliore**: v2, la sua misura, il confronto, la riga

Ogni misura, due chiamate brevi senza strumenti: la risposta su Sonnet, il giudizio su Haiku. **Opus** pesa di più sul limite d'uso: per una misura sola. Niente Fable

Checkpoint: la prima famiglia misurata fino a v2, e annotata nel registro

Laboratorio: secondo prompt

La seconda famiglia, allo stesso modo — e il giudice si legge

- La **seconda famiglia**, allo stesso modo: `v0`, `v1`, `v2`, i confronti e le righe del registro
- Il giudice è anch'esso **un modello**: leggete i **motivi**; quando non siete d'accordo **decidete voi** — e se serve riscrivete l'affermazione della rubrica perché sia verificabile senza ambiguità
- Rilanciando la stessa versione, risposta e giudizio **possono cambiare**
- Se qualcosa va storto: un segnaposto che in `dati/` non c'è — controllate il nome fra le doppie graffe; un giudizio senza esito strutturato — rilanciate la misura della stessa versione; una famiglia che peggiora — tornate alla **versione migliore**; per ricominciare da capo, `tools/reset_esercizio.sh M2-E02`

Checkpoint: la seconda famiglia misurata fino a v2, e annotata nel registro

Laboratorio: terzo prompt

L'ultima famiglia, poi la verifica

- La **terza famiglia**, allo stesso modo — se il tempo stringe, basta **un ciclo**: `v0` e `v1`
- Alla fine, `/exit`; `cd -` riporta nella cartella `esercitazioni`, e da lì la verifica:

```
# in Claude Code /exit, poi:
cd -
tools/verifica.sh M2-E02    # 0 = fatto, 1 = non ancora
```

- La verifica controlla: in ogni famiglia `v0` e `v1` misurate, e `v1.md` diverso da `v0.md`; sulla somma di controlli superati e affermazioni rispettate, l'ultima versione contro `v0`, **nessuna** famiglia peggiora e **almeno due** migliorano; almeno due famiglie con v2; le righe di `v0` e `v1` compilate in `registro.md`
- Il giudizio lo dà un modello: se una famiglia non migliora, **un altro ciclo** è la risposta

Checkpoint: la terza famiglia misurata e annotata nel registro — e la verifica esce con 0